

UNIVERSIDAD DE NARIÑO

Especialización en Construcción de Software

Guía 2: Integración: PostgreSQL+MinIO+Data Lakes

Estructura de Directorios

```
/home/<su_usuario>/
├── projects/
│   ├── ebd_env/                # Entorno virtual Python
│   ├── notebooks/             # Notebooks Jupyter
│   │   ├── 01_verificacion.ipynb
│   │   ├── 02_etl_postgres.ipynb
│   │   └── 03_minio_datalake.ipynb
│   ├── data/                  # Datos de práctica
│   │   ├── clientes.csv
│   │   └── ventas.csv
│   ├── jars/                  # JARs (Delta Lake, PostgreSQL)
│   └── minio/
│       ├── bin/minio           # Binario MinIO
│       ├── data/               # Storage persistente
│       ├── start_minio.sh      # Script de inicio
│       └── start_jupyter.sh
└──
```

EQUIVALENCIAS CON SERVICIOS CLOUD

MinIO (local)	→ Amazon S3 / Azure Blob Storage / Google Cloud Storage
Protocolo s3a://	→ s3a:// (AWS) / abfss:// (Azure) / gs:// (GCP)
Spark Standalone	→ AWS EMR / Azure Synapse / Databricks / GCP Dataproc
Delta Lake (local)	→ Delta Lake (cloud) / Apache Iceberg
PostgreSQL (local)	→ Amazon RDS / Azure SQL / Cloud SQL

Instalación Paso a Paso

1. Instalar PostgreSQL 15

```
Shell

# Actualizar repositorios
sudo apt update

# Instalar PostgreSQL 15 y utilidades
sudo apt install postgresql postgresql-contrib -y

# Verificar versión y estado del servicio
```

```
psql --version
sudo systemctl status postgresql
```

Salida esperada:

```
Shell
psql (PostgreSQL) 16.14 (Ubuntu 16.14-0ubuntu0.24.04.1)
• postgresql.service - PostgreSQL RDBMS
  Loaded: loaded (...)
  Active: active (running) since ...
```

2. Copiar archivos .sql

```
Shell
# Descargar y Descomprimir archivo .zip con archivos .sql

# Copiar archivo .sql a /var/lib/postgresql/
sudo cp <ruta>/db_electiva.sql /var/lib/postgresql/
sudo cp <ruta>/inserts.sql /var/lib/postgresql/
```

3. Configuration PostgreSQL

```
Shell
# Cambiar al usuario postgres
sudo -u postgres psql
```

Dentro del prompt **postgres=#**, ejecutar:

```
SQL
-- Crear usuario (ya existe 'postgres', solo verificamos password)
ALTER USER postgres WITH PASSWORD '123abc';

-- Crear base de datos del electiva
CREATE DATABASE electiva;

-- Conectar a la base de datos datamart
\c electiva

-- Crear tablas: clientes y ventas en base de datos electiva
\i /var/lib/postgresql/tables.sql

-- Crear tablas: clientes y ventas en base de datos electiva
```

```
\i /var/lib/postgresql/inserts.sql
```

```
-- Verificar datos
```

```
SELECT * FROM clientes;
```

id_cliente	nombre	email	ciudad	fecha_registro	segmento
1	Ana García	ana.garcia@email.com	Bogotá	2023-01-15	Premium
2	Luis Martínez	luis.m@email.com	Medellín	2023-03-22	Estándar
3	Carla Rojas	carla.r@email.com	Cali	2023-06-10	Premium
4	Pedro Sánchez	pedro.s@email.com	Barranquilla	2023-08-05	Básico
5	Maria López	maria.l@email.com	Bogotá	2023-09-12	Estándar

```
(5 rows)
```

```
SELECT * FROM ventas;
```

id_venta	id_cliente	producto	cantidad	precio_unitario	fecha_venta	total
1	1	Laptop Pro	1	2500.00	2024-01-10	2500.00
2	1	Mouse Inalámbrico	2	25.50	2024-01-15	51.00
3	2	Teclado Mecánico	1	120.00	2024-02-20	120.00
4	3	Monitor 4K	2	450.00	2024-03-05	900.00
5	3	Webcam HD	1	80.00	2024-03-10	80.00
6	4	Audífonos Bluetooth	3	60.00	2024-04-01	180.00
7	5	Disco Duro Externo	1	150.00	2024-04-15	150.00

```
(7 rows)
```

```
-- Salir
```

```
\q
```

4. Configuración Acceso Remoto/Local

```
Shell
```

```
# Editar configuración de PostgreSQL
```

```
sudo nano /etc/postgresql/16/main/pg_hba.conf
```

Asegurarse de que exista esta línea (modo **md5** para password):

```
Shell
```

```
# IPv4 local connections:
```

host	all	all	127.0.0.1/32	md5
------	-----	-----	--------------	-----

Ejecute las siguientes líneas en la terminal

```
Shell
```

```
# Reiniciar PostgreSQL
```

```
sudo systemctl restart postgresql
```

```
# Verificar que escucha en puerto 5432
```

```
sudo netstat -tlnp | grep 5432
# o
sudo ss -tlnp | grep 5432
```

5. Verificar Conectividad JDBC

```
Shell
# Desde terminal, probar con psql
psql -h localhost -p 5432 -U postgres -d electiva

# Ingresar password: 123abc
# Debe conectar exitosamente
psql (16.14 (Ubuntu 16.14-0ubuntu0.24.04.1))
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off)
Type "help" for help.

electiva=#

# Salir
\q
```

6. Instalación y Configuración de MinIO

Descargar e Instalar MinIO

```
Shell
# Crear estructura de directorios
mkdir -p ~/projects/minio
mkdir -p ~/projects/minio/{bin,data}
cd ~/projects/minio

# Descargar binario de MinIO (versión standalone)
wget https://dl.min.io/server/minio/release/linux-amd64/minio

# Mover al directorio bin
mv minio ~/projects/minio/bin/

# Entrar al directorio bin
cd ~/projects/minio/bin/

# Dar permisos de ejecución
chmod +x minio

# Verificar
```

```
./bin/minio --version
```

```
minio version RELEASE.2025-09-07T16-13-09Z  
(commit-id=07c3a429bfed433e49018cb0f78a52145d4bedeb)  
Runtime: go1.24.6 linux/amd64  
License: GNU AGPLv3 - https://www.gnu.org/licenses/agpl-3.0.html  
Copyright: 2015-2025 MinIO, Inc.
```

7. Crear Script de Inicio

Shell

```
cd ~/projects/minio  
cat > start_minio.sh << 'EOF'  
#!/bin/bash  
  
# Script de inicio de MinIO  
# Puerto 9000: API S3  
# Puerto 9001: Consola Web  
  
export MINIO_ROOT_USER=minioadmin  
export MINIO_ROOT_PASSWORD=minioadmin123  
  
~/minio/bin/minio server ~/minio/data --console-address ":9001"  
EOF  
  
chmod +x start_minio.sh
```

8. Iniciar Minlo

Shell

```
# Ejecutar en terminal (dejar corriendo, o usar screen/tmux)  
cd ~/projects/minio  
./start_minio.sh
```

Salida esperando

Shell

```
INFO: Formatting 1st pool, 1 set(s), 1 drives per set.  
INFO: WARNING: Host local has more than 0 drives of set. A host failure will result in data  
becoming unavailable.  
MinIO Object Storage Server  
Copyright: 2015-2026 MinIO, Inc.  
License: GNU AGPLv3 - https://www.gnu.org/licenses/agpl-3.0.html
```

```
Version: RELEASE.2025-09-07T16-13-09Z (go1.24.6 linux/amd64)

API: http://192.168.231.129:9000 http://127.0.0.1:9000
    RootUser: minioadmin
    RootPass: minioadmin123

WebUI: http://192.168.231.129:9001 http://127.0.0.1:9001
    RootUser: minioadmin
    RootPass: minioadmin123

CLI: https://docs.min.io/community/minio-object-store/reference/minio-mc.html#quickstart
    $ mc alias set 'myminio' 'http://192.168.231.129:9000' 'minioadmin' 'minioadmin123'

Docs: https://docs.min.io
```

Nota: Para mantener MinIO corriendo en segundo plano, abre otra terminal o usa:

```
Shell
nohup ~/minio/start_minio.sh > ~/minio/minio.log 2>&1 &
```

9. Acceder a la Consola Web

- a. Abrir el navegador en la VM: <http://localhost:9001>
- b. Login:
 - i. Usuario: minioadmin
 - ii. Contraseña: minioadmin123
- c. Crear bucket:
 - i. Ir a Buckets → Create Bucket
 - ii. Nombre: spark-lake
 - iii. Guardar

10. Verificar JARs de Hadoop AWS (ya descargados en Guía 1)

```
Shell
# Verificar que los JARs están en spark/jars/
ls -la /usr/local/spark-3.5.8/jars/ | grep -E "hadoop-aws|aws-java-sdk|postgresql"

-rw-r--r--  1 root root 280645251 Jun  4 22:30 aws-java-sdk-bundle-1.12.262.jar
-rw-r--r--  1 root root   962685 Jun  4 22:30 hadoop-aws-3.3.4.jar
-rw-r--r--  1 root root  1089312 Jun  4 22:30 postgresql-42.7.3.jar
```

Notebook de verificación - ETL POSTGRESQL

Objetivo:

Leer desde PostgreSQL, transformar con Spark y escribir de vuelta a PostgreSQL.

Asegúrese de estar en directorio projects

Shell

```
# Entrar a la carpeta projects
cd ~/projects

# Inicie Notebook de Jupyter
./start_jupyter.sh
```

1. Celda 1: Configuración de Conexión JDBC

Python

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, when, lit, current_date, sum as spark_sum,
count, avg

# Configuración de conexión PostgreSQL
jdbc_url = "jdbc:postgresql://localhost:5432/electiva"

connection_properties = {
    "user": "postgres",
    "password": "123abc",
    "driver": "org.postgresql.Driver"
}

print("Configuración JDBC definida:")
print(f"URL: {jdbc_url}")
print(f"Driver: {connection_properties['driver']}")
```

Salida esperada:

Shell

```
Configuración JDBC definida:
URL: jdbc:postgresql://localhost:5432/electiva
Driver: org.postgresql.Driver
```

2. Celda 2: Crear SparkSession con soporte JDBC

Python

```
spark = (SparkSession.builder
        .appName("ETL_PostgreSQL")
        .master("local[*]")
        .config("spark.driver.memory", "1g")
        .config("spark.executor.memory", "1500m")
        .config("spark.jars",
        "/usr/local/spark-3.5.8/jars/postgresql-42.7.3.jar")
        .getOrCreate())

print("SparkSession creada con soporte JDBC")
print(f"App: {spark.sparkContext.appName}")
print(f"Version: {spark.version}")
```

Salida esperada:

Shell

```
SparkSession creada con soporte JDBC
App: ETL_PostgreSQL
Version: 3.5.8
```

3. Celda 3: Leer Datos desde PostgreSQL

Python

```
print("=" * 60)
print("LECTURA DESDE POSTGRESQL")
print("=" * 60)

# Leer tabla clientes
df_clientes = spark.read.jdbc(
    url=jdbc_url,
    table="clientes",
    properties=connection_properties
)

print("Tabla 'clientes':")
df_clientes.printSchema()
df_clientes.show()

# Leer tabla ventas
df_ventas = spark.read.jdbc(
    url=jdbc_url,
```



```

        table="ventas",
        properties=connection_properties
    )

    print("\nTabla 'ventas':")
    df_ventas.printSchema()
    df_ventas.show()

```

Salida esperada:

Shell

```

=====
LECTURA DESDE POSTGRESQL
=====

```

Tabla 'clientes':

root

```

|-- id_cliente: integer (nullable = true)
|-- nombre: string (nullable = true)
|-- email: string (nullable = true)
|-- ciudad: string (nullable = true)
|-- fecha_registro: date (nullable = true)
|-- segmento: string (nullable = true)

```

id_cliente	nombre	email	ciudad	fecha_registro	segmento
1	Ana García	ana.garcia@email.com	Bogotá	2023-01-15	Premium
2	Luis Martínez	luis.m@email.com	Medellín	2023-03-22	Estándar
3	Carla Rojas	carla.r@email.com	Cali	2023-06-10	Premium
4	Pedro Sánchez	pedro.s@email.com	Barranquilla	2023-08-05	Básico
5	María López	maria.l@email.com	Bogotá	2023-09-12	Estándar

Tabla 'ventas':

root

```

|-- id_venta: integer (nullable = true)
|-- id_cliente: integer (nullable = true)
|-- producto: string (nullable = true)
|-- cantidad: integer (nullable = true)
|-- precio_unitario: decimal(10,2) (nullable = true)
|-- fecha_venta: date (nullable = true)
|-- total: decimal(10,2) (nullable = true)

```

id_venta	id_cliente	producto	cantidad	precio_unitario	fecha_venta	total
1	1	Laptop Pro	1	2500.00	2024-01-10	2500.00
2	1	Mouse Inalámbrico	2	25.50	2024-01-15	51.00
3	2	Teclado Mecánico	1	120.00	2024-02-20	120.00
4	3	Monitor 4K	2	450.00	2024-03-05	900.00
5	3	Webcam HD	1	80.00	2024-03-10	80.00
6	4	Audífonos Bluetooth	3	60.00	2024-04-01	180.00
7	5	Disco Duro Externo	1	150.00	2024-04-15	150.00

4. Celda 4: Transformaciones con DataFrame

Python

```
print("=" * 60)
print("TRANSFORMACIONES")
print("=" * 60)

# Join entre clientes y ventas
df_joined = df_clientes.join(
    df_ventas,
    df_clientes.id_cliente == df_ventas.id_cliente,
    "inner"
).select(
    df_clientes.id_cliente,
    "nombre",
    "ciudad",
    "segmento",
    "producto",
    "cantidad",
    "precio_unitario",
    "total",
    "fecha_venta"
)

print("Join Clientes + Ventas:")
df_joined.show()

# Agregaciones por segmento
df_por_segmento = df_joined.groupBy("segmento").agg(
    count("*").alias("total_ventas"),
    spark_sum("total").alias("ingresos_totales"),
    avg("total").alias("ticket_promedio")
)
```

```

)

print("\nAgregación por segmento:")
df_por_segmento.show()

# Clientes Premium con alto valor
df_premium = df_joined.filter(
    (col("segmento") == "Premium") & (col("total") > 100)
).withColumn(
    "categoria_valor",
    when(col("total") > 1000, "Alto Valor")
    .when(col("total") > 500, "Medio Valor")
    .otherwise("Bajo Valor")
)

print("\nClientes Premium con categorización:")
df_premium.show()

```

Salida esperada:

Shell

TRANSFORMACIONES

Join Clientes + Ventas:

id_cliente	nombre	ciudad	segmento	producto	cantidad	precio_unitario	total	fecha_venta
	1	Ana García	Bogotá	Premium	Laptop Pro	1	2500.00 2500.00	2024-01-10
	1	Ana García	Bogotá	Premium	Mouse Inalámbrico	2	25.50 51.00	2024-01-15
	2	Luis Martínez	Medellín	Estándar	Teclado Mecánico	1	120.00 120.00	2024-02-20
	3	Carla Rojas	Cali	Premium	Monitor 4K	2	450.00 900.00	2024-03-05
	3	Carla Rojas	Cali	Premium	Webcam HD	1	80.00 80.00	2024-03-10
	4	Pedro Sánchez	Barranquilla	Básico	Audífonos Bluetooth	3	60.00 180.00	2024-04-01
	5	María López	Bogotá	Estándar	Disco Duro Externo	1	150.00 150.00	2024-04-15

Agregación por segmento:

segmento	total_ventas	ingresos_totales	ticket_promedio
Premium	4	3531.00	882.750000
Estándar	2	270.00	135.000000
Básico	1	180.00	180.000000

Cientes Premium con categorización:

id_cliente	nombre	ciudad	segmento	producto	cantidad	precio_unitario	total	fecha_venta	categoria_valor
1	Ana García	Bogotá	Premium	Laptop Pro	1	2500.00	2500.00	2024-01-10	Alto Valor
3	Carla Rojas	Cali	Premium	Monitor 4K	2	450.00	900.00	2024-03-05	Medio Valor

5. Celda 5: Escritura a PostgreSQL (Modo Append)

Python

```
from pyspark.sql.types import Datetype

print("=" * 60)
print("ESCRITURA A POSTGRESQL (APPEND)")
print("=" * 60)

# Crear tabla resumen de ventas por cliente
df_resumen = df_joined.groupBy("id_cliente", "nombre", "ciudad", "segmento").agg(
    count("*").alias("num_compras"),
    spark_sum("total").alias("total_gastado"),
    avg("total").alias("promedio_compra")
).withColumn("fecha_analisis", current_date().cast(DateType()))

print("DataFrame a escribir (resumen_clientes):")
df_resumen.printSchema()
df_resumen.show()

# Escribir en PostgreSQL (modo overwrite para crear tabla)
df_resumen.write.jdbc(
    url=jdbc_url,
    table="resumen_clientes",
    mode="overwrite", # Crea o reemplaza la tabla
    properties=connection_properties
)

print("Tabla 'resumen_clientes' creada en PostgreSQL")
```

Salida esperada:

Shell

```
=====
ESCRITURA A POSTGRESQL (APPEND)
```

=====

DataFrame a escribir (resumen_clientes):

root

```
|-- id_cliente: integer (nullable = true)
|-- nombre: string (nullable = true)
|-- ciudad: string (nullable = true)
|-- segmento: string (nullable = true)
|-- num_compras: long (nullable = false)
|-- total_gastado: decimal(20,2) (nullable = true)
|-- promedio_compra: decimal(14,6) (nullable = true)
|-- fecha_analisis: date (nullable = false)
```

id_cliente	nombre	ciudad	segmento	num_compras	total_gastado	promedio_compra	fecha_analisis
1	Ana García	Bogotá	Premium	2	2551.00	1275.500000	2026-06-09
2	Luis Martínez	Medellín	Estándar	1	120.00	120.000000	2026-06-09
3	Carla Rojas	Cali	Premium	2	980.00	490.000000	2026-06-09
4	Pedro Sánchez	Barranquilla	Básico	1	180.00	180.000000	2026-06-09
5	Maria López	Bogotá	Estándar	1	150.00	150.000000	2026-06-09

Tabla 'resumen_clientes' creada en PostgreSQL

6. Celda 6: Verificar Escritura y Lectura de Vuelta

Python

```
print("=" * 60)
print("APPEND DE DATOS ADICIONALES (CON DATE EXPLÍCITO)")
print("=" * 60)

from datetime import date

# Crear DataFrame con fecha como DateType (NO como string)
from pyspark.sql import Row

# Usar Row con fecha como objeto date de Python
df_append = spark.createDataFrame([
    Row(
        id_cliente=99,
        nombre="Cliente Nuevo",
        ciudad="Cartagena",
        segmento="Estándar",
        num_compras=2,
        total_gastado=300.00,
```

```

        promedio_compra=150.00,
        fecha_analisis=date(2024, 5, 20) # ← date de Python, no string
    )
])

# Verificar esquema antes de escribir
print("Esquema del DataFrame a insertar:")
df_append.printSchema()
df_append.show()

# Escribir en modo append
df_append.write.jdbc(
    url=jdbc_url,
    table="resumen_clientes",
    mode="append",
    properties=connection_properties
)

print("✅ Datos adicionales insertados en modo APPEND")

# Verificar
df_final = spark.read.jdbc(
    url=jdbc_url,
    table="resumen_clientes",
    properties=connection_properties
)

print("\nTabla final con datos append:")
df_final.show()
df_final.printSchema()

```

Salida esperada:

```

Shell
=====
APPEND DE DATOS ADICIONALES (CON DATE EXPLÍCITO)
=====
Esquema del DataFrame a insertar:
root
|-- id_cliente: long (nullable = true)
|-- nombre: string (nullable = true)
|-- ciudad: string (nullable = true)
|-- segmento: string (nullable = true)
|-- num_compras: long (nullable = true)

```

```
|-- total_gastado: double (nullable = true)
|-- promedio_compra: double (nullable = true)
|-- fecha_analisis: date (nullable = true)
```

id_cliente	nombre	ciudad	segmento	num_compras	total_gastado	promedio_compra	fecha_analisis
99	Cliente Nuevo	Cartagena	Estándar	2	300.0	150.0	2024-05-20

Datos adicionales insertados en modo APPEND

Tabla final con datos append:

id_cliente	nombre	ciudad	segmento	num_compras	total_gastado	promedio_compra	fecha_analisis
1	Ana García	Bogotá	Premium	2	2551.00	1275.500000	2026-06-10
2	Luis Martínez	Medellín	Estándar	1	120.00	120.000000	2026-06-10
3	Carla Rojas	Cali	Premium	2	980.00	490.000000	2026-06-10
4	Pedro Sánchez	Barranquilla	Básico	1	180.00	180.000000	2026-06-10
5	María López	Bogotá	Estándar	1	150.00	150.000000	2026-06-10
99	Cliente Nuevo	Cartagena	Estándar	2	300.00	150.000000	2024-05-20

root

```
|-- id_cliente: integer (nullable = true)
|-- nombre: string (nullable = true)
|-- ciudad: string (nullable = true)
|-- segmento: string (nullable = true)
|-- num_compras: long (nullable = true)
|-- total_gastado: decimal(20,2) (nullable = true)
|-- promedio_compra: decimal(14,6) (nullable = true)
|-- fecha_analisis: date (nullable = true)
```

7. Manejo de Tipos de Datos (DATE)

Python

```
print("=" * 60)
print("MANEJO DE TIPOS DE DATOS - DATE EN POSTGRESQL")
print("=" * 60)
```

```
# Verificar tipos de fecha en el DataFrame original
print("Esquema de ventas (tipos originales):")
df_ventas.printSchema()
```

```
# Convertir fecha_venta a DateType explícito
```

```

df_ventas_fecha = df_ventas.withColumn("fecha_venta",
col("fecha_venta").cast(DateType()))

print("\nEsquema con fecha convertida:")
df_ventas_fecha.printSchema()

# Filtrar por rango de fechas
df_marzo = df_ventas_fecha.filter(
    (col("fecha_venta") >= "2024-03-01") &
    (col("fecha_venta") <= "2024-03-31")
)

print("\nVentas de marzo 2024:")
df_marzo.show()

# Escribir particionado por fecha (sobrescribir tabla de prueba)
df_ventas_fecha.write \
    .option("createTableColumnTypes", "fecha_venta DATE") \
    .jdbc(url=jdbc_url, table="ventas_particionadas", mode="overwrite",
properties=connection_properties)

print("✅ Datos escritos con tipo DATE explícito en PostgreSQL")

# Verificar tipos en PostgreSQL
df_verificacion_pg = spark.read.jdbc(
    url=jdbc_url,
    table="ventas_particionadas",
    properties=connection_properties
)

print("\nEsquema leído desde PostgreSQL:")
df_verificacion_pg.printSchema()

```

Salida esperada:

```

Shell
=====
MANEJO DE TIPOS DE DATOS - DATE EN POSTGRESQL
=====
Esquema de ventas (tipos originales):
root
|-- id_venta: integer (nullable = true)
|-- id_cliente: integer (nullable = true)
|-- producto: string (nullable = true)

```



```
|-- cantidad: integer (nullable = true)
|-- precio_unitario: decimal(10,2) (nullable = true)
|-- fecha_venta: date (nullable = true)
|-- total: decimal(10,2) (nullable = true)
```

Esquema con fecha convertida:

```
root
|-- id_venta: integer (nullable = true)
|-- id_cliente: integer (nullable = true)
|-- producto: string (nullable = true)
|-- cantidad: integer (nullable = true)
|-- precio_unitario: decimal(10,2) (nullable = true)
|-- fecha_venta: date (nullable = true)
|-- total: decimal(10,2) (nullable = true)
```

Ventas de marzo 2024:

id_venta	id_cliente	producto	cantidad	precio_unitario	fecha_venta	total
4	3	Monitor 4K	2	450.00	2024-03-05	900.00
5	3	Webcam HD	1	80.00	2024-03-10	80.00

Datos escritos con tipo DATE explícito en PostgreSQL

Esquema leído desde PostgreSQL:

```
root
|-- id_venta: integer (nullable = true)
|-- id_cliente: integer (nullable = true)
|-- producto: string (nullable = true)
|-- cantidad: integer (nullable = true)
|-- precio_unitario: decimal(10,2) (nullable = true)
|-- fecha_venta: date (nullable = true)
|-- total: decimal(10,2) (nullable = true)
```

Notebook de verificación - MINIO DATA LAKE

Objetivo:

Integrar Spark con MinIO (S3 local) para operaciones de Data Lake: lectura/escritura de Delta Lake en buckets S3.

1. Verificar MinIO activo

Python

```
import subprocess
import os

print("=" * 60)
print("VERIFICACIÓN DE MINIO")
print("=" * 60)

# Verificar que MinIO está corriendo
result = subprocess.run(["pgrep", "-f", "minio"], capture_output=True, text=True)
if result.returncode == 0:
    print("Proceso MinIO encontrado")
else:
    print("MinIO no está corriendo. Ejecute ~/minio/start_minio.sh")

# Verificar puertos
for port in [9000, 9001]:
    result = subprocess.run(["nc", "-z", "localhost", str(port)],
capture_output=True)
    status = "Abierto" if result.returncode == 0 else "Cerrado"
    print(f" Puerto {port}: {status}")

print("\nConfiguración S3:")
print("  Endpoint: http://localhost:9000")
print("  Access Key: minioadmin")
print("  Secret Key: minioadmin123")
print("  Bucket: spark-lake")
```

Salida esperada:

Shell

```
=====
VERIFICACIÓN DE MINIO
=====
Proceso MinIO encontrado
  Puerto 9000: Abierto
  Puerto 9001: Abierto
```

Configuración S3:

Endpoint: http://localhost:9000
Access Key: minioadmin
Secret Key: minioadmin123
Bucket: spark-lake

2. Verificar MinIO activo

Python

```
from pyspark.sql import SparkSession

print("=" * 60)
print("CREACIÓN DE SPARKSESSION CON SOPORTE S3A")
print("=" * 60)

spark = (SparkSession.builder
        .appName("MinIO_DataLake")
        .master("local[*]")
        .config("spark.driver.memory", "1g")
        .config("spark.executor.memory", "1500m")
        # Configuración S3A para MinIO
        .config("spark.hadoop.fs.s3a.endpoint", "http://localhost:9000")
        .config("spark.hadoop.fs.s3a.access.key", "minioadmin")
        .config("spark.hadoop.fs.s3a.secret.key", "minioadmin123")
        .config("spark.hadoop.fs.s3a.path.style.access", "true")
        .config("spark.hadoop.fs.s3a.impl",
"org.apache.hadoop.fs.s3a.S3AFileSystem")
        .config("spark.hadoop.fs.s3a.connection.ssl.enabled", "false")
        # Delta Lake
        .config("spark.sql.extensions",
"io.delta.sql.DeltaSparkSessionExtension")
        .config("spark.sql.catalog.spark_catalog",
"org.apache.spark.sql.delta.catalog.DeltaCatalog")
        .getOrCreate())

print("SparkSession creada con soporte S3A/MinIO")
print(f"    App: {spark.sparkContext.appName}")
```

Salida esperada:

Shell

=====

CREACIÓN DE SPARKSESSION CON SOPORTE S3A

=====

SparkSession creada con soporte S3A/MinIO

App: MinIO_DataLake

3. Verificar Bucket con Boto3

Python

```
import boto3
from botocore.exceptions import ClientError

print("=" * 60)
print("VERIFICACIÓN DE BUCKET CON BOTO3")
print("=" * 60)

# Cliente S3 apuntando a MinIO
s3 = boto3.client(
    's3',
    endpoint_url='http://localhost:9000',
    aws_access_key_id='minioadmin',
    aws_secret_access_key='minioadmin123',
    region_name='us-east-1'
)

# Listar buckets
try:
    response = s3.list_buckets()
    print("Buckets disponibles:")
    for bucket in response['Buckets']:
        print(f"    - {bucket['Name']} (creado: {bucket['CreationDate']})")
except ClientError as e:
    print(f" Error: {e}")

# Crear bucket si no existe
bucket_name = "spark-lake"
try:
    s3.head_bucket(Bucket=bucket_name)
    print(f"\nBucket '{bucket_name}' ya existe")
except ClientError:
    s3.create_bucket(Bucket=bucket_name)
    print(f"\nBucket '{bucket_name}' creado")
```

Salida esperada:

Shell

```
=====
VERIFICACIÓN DE BUCKET CON BOT03
=====
Buckets disponibles:
- spark-lake (creado: 2026-06-10 02:26:09.962000+00:00)

Bucket 'spark-lake' ya existe
```

4. Crear Datos de Prueba y Escribir a MinIO (Delta)

Python

```
from pyspark.sql.functions import col, lit, current_date, rand, when
from pyspark.sql.types import StructType, StructField, StringType, IntegerType,
DoubleType, DateType

print("=" * 60)
print("ESCRITURA DE DELTA LAKE A MINIO")
print("=" * 60)

# Crear DataFrame de transacciones
data = [
    ("TXN001", "C001", "Laptop", 1, 2500.00, "2024-01-15"),
    ("TXN002", "C002", "Mouse", 3, 25.50, "2024-01-16"),
    ("TXN003", "C001", "Teclado", 1, 120.00, "2024-01-17"),
    ("TXN004", "C003", "Monitor", 2, 450.00, "2024-01-18"),
    ("TXN005", "C002", "Webcam", 1, 80.00, "2024-01-19"),
]

schema = StructType([
    StructField("id_transaccion", StringType(), False),
    StructField("id_cliente", StringType(), False),
    StructField("producto", StringType(), False),
    StructField("cantidad", IntegerType(), False),
    StructField("precio", DoubleType(), False),
    StructField("fecha", StringType(), False)
])

df_transacciones = spark.createDataFrame(data, schema)
df_transacciones = df_transacciones.withColumn("fecha",
col("fecha").cast(DateType()))

print("DataFrame de transacciones:")
df_transacciones.show()
```

```
# Ruta S3A
delta_path = "s3a://spark-lake/transacciones"

print(f"\nEscribiendo en: {delta_path}")
df_transacciones.write.format("delta").mode("overwrite").save(delta_path)
print("Datos escritos en Delta Lake sobre MinIO")
```

Salida esperada:

Shell

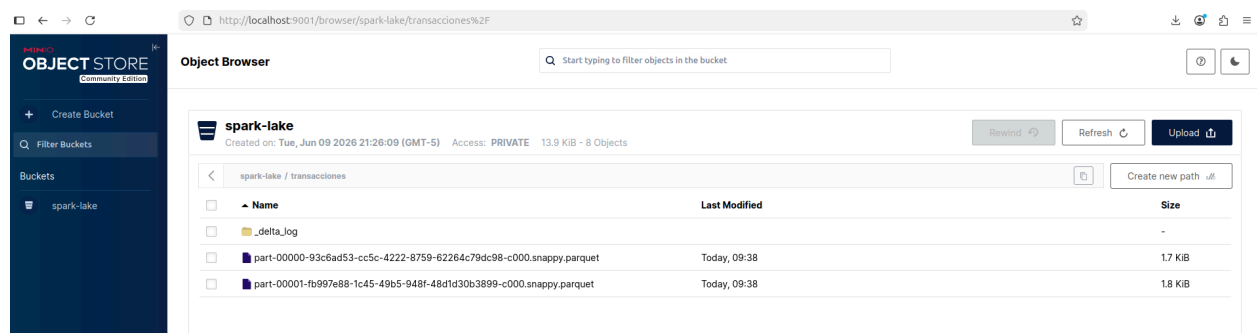
```
=====
ESCRITURA DE DELTA LAKE A MINIO
=====
```

DataFrame de transacciones:

id_transaccion	id_cliente	producto	cantidad	precio	fecha
TXN001	C001	Laptop	1	2500.0	2024-01-15
TXN002	C002	Mouse	3	25.5	2024-01-16
TXN003	C001	Teclado	1	120.0	2024-01-17
TXN004	C003	Monitor	2	450.0	2024-01-18
TXN005	C002	Webcam	1	80.0	2024-01-19

Escribiendo en: s3a://spark-lake/transacciones

Datos escritos en Delta Lake sobre MinIO



5. Leer desde MinIO y Transformar

Python

```
print("=" * 60)
print("LECTURA Y TRANSFORMACIÓN DESDE MINIO")
print("=" * 60)

# Leer desde Delta en MinIO
df_delta = spark.read.format("delta").load(delta_path)
print("Datos leídos desde Delta Lake/MinIO:")
df_delta.show()

# Transformaciones
df_transformado = df_delta.withColumn(
    "total_linea", col("cantidad") * col("precio")
).withColumn(
    "categoria",
    when(col("total_linea") > 1000, "Alto Valor")
    .when(col("total_linea") > 100, "Medio Valor")
    .otherwise("Bajo Valor")
).withColumn("fecha_procesamiento", current_date())

print("\nDataFrame transformado:")
df_transformado.show()

# Escribir resultado transformado
delta_transformado = "s3a://spark-lake/transacciones_transformadas"
df_transformado.write.format("delta").mode("overwrite").save(delta_transformado)
print(f"\nDatos transformados escritos en: {delta_transformado}")
```

Salida esperada:

Shell

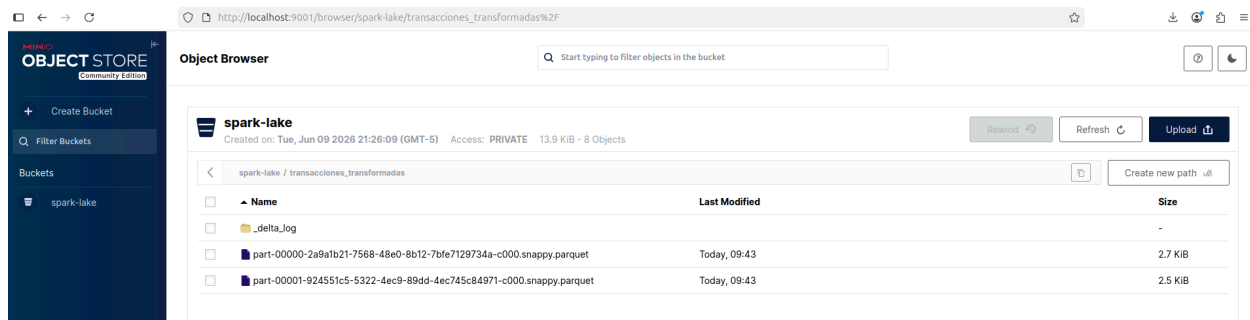
```
=====
LECTURA Y TRANSFORMACIÓN DESDE MINIO
=====
Datos leídos desde Delta Lake/MinIO:

+-----+-----+-----+-----+-----+-----+
|id_transaccion|id_cliente|producto|cantidad|precio|      fecha|
+-----+-----+-----+-----+-----+-----+
|      TXN003 |      C001| Teclado|        1|  120.0|2024-01-17|
|      TXN004 |      C003| Monitor|        2|  450.0|2024-01-18|
|      TXN005 |      C002| Webcam |        1|   80.0|2024-01-19|
|      TXN001 |      C001| Laptop |       1|2500.0|2024-01-15|
|      TXN002 |      C002| Mouse  |        3|   25.5|2024-01-16|
+-----+-----+-----+-----+-----+-----+
```

DataFrame transformado:

id_transaccion	id_cliente	producto	cantidad	precio	fecha	total_linea	categoria	fecha_procesamiento
TXN003	C001	Teclado	1	120.0	2024-01-17	120.0	Medio Valor	2026-06-10
TXN004	C003	Monitor	2	450.0	2024-01-18	900.0	Medio Valor	2026-06-10
TXN005	C002	Webcam	1	80.0	2024-01-19	80.0	Bajo Valor	2026-06-10
TXN001	C001	Laptop	1	2500.0	2024-01-15	2500.0	Alto Valor	2026-06-10
TXN002	C002	Mouse	3	25.5	2024-01-16	76.5	Bajo Valor	2026-06-10

Datos transformados escritos en: s3a://spark-lake/transacciones_transformadas



6. Time Travel con Delta Lake en MinIO

Python

```
from delta import DeltaTable

print("=" * 60)
print("TIME TRAVEL CON DELTA LAKE")
print("=" * 60)

# Ver historial
delta_table = DeltaTable.forPath(spark, delta_path)
print("Historial de versiones:")
delta_table.history().show(truncate=False)

# Agregar nuevos datos (nueva versión)
nuevas_transacciones = [
    ("TXN006", "C004", "Audífonos", 2, 60.00, "2024-01-20"),
    ("TXN007", "C005", "Disco Duro", 1, 150.00, "2024-01-21"),
]

df_nuevas = spark.createDataFrame(nuevas_transacciones, schema)
df_nuevas = df_nuevas.withColumn("fecha", col("fecha").cast(DateType()))
```



```

# Merge (UPSERT) - actualizar si existe, insertar si no
delta_table.alias("target").merge(
    df_nuevas.alias("source"),
    "target.id_transaccion = source.id_transaccion"
).whenMatchedUpdateAll().whenNotMatchedInsertAll().execute()

print("Merge ejecutado (UPSERT)")

# Ver nuevo historial
print("\nNuevo historial:")
delta_table.history().show(truncate=False)

# Leer versión anterior
df_version_0 = spark.read.format("delta").option("versionAsOf",
0).load(delta_path)
print("\nVersión 0 (original):")
df_version_0.show()

df_version_1 = spark.read.format("delta").option("versionAsOf",
1).load(delta_path)
print("\nVersión 1 (después del merge):")
df_version_1.show()

```

Salida esperada:

Shell

```

=====
TIME TRAVEL CON DELTA LAKE
=====
Historial de versiones:

+-----+-----+-----+-----+-----+-----+
|version|timestamp|userId|userName|operation|operationParameters| |
|job |notebook|clusterId|readVersion|isolationLevel|isBlindAppend|operationMetrics|
|userMetadata|engineInfo|
+-----+-----+-----+-----+-----+-----+
|0|2026-06-10 09:39:00|NULL|NULL|WRITE|{mode -> Overwrite,
partitionBy -> []}|NULL|NULL|NULL|Serializable|false

```

```
|{numFiles -> 2, numOutputRows -> 5, numOutputBytes -> 3552}|NULL
```

```
|Apache-Spark/3.5.8 Delta-Lake/3.2.0|
```

```
+-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+
```

Merge ejecutado (UPSERT)

Nuevo historial:

```
+-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+
```

```
|version|timestamp          |userId|userName|operation|operationParameters
|job   |notebook|clusterId|readVersion|isolationLevel|isBlindAppend|operationMetrics
|userMetadata|engineInfo          |
```

```
+-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+
```

```
|1      |2026-06-10 09:54:33|NULL  |NULL    |MERGE    |{predicate ->
["(id_transaccion#2016 = id_transaccion#2184)"], matchedPredicates ->
[{"actionType":"update"}], notMatchedPredicates -> [{"actionType":"insert"}],
notMatchedBySourcePredicates -> []}|NULL|NULL    |NULL    |0
|Serializable |false          |{numTargetRowsCopied -> 0, numTargetRowsDeleted ->
0, numTargetFilesAdded -> 2, numTargetBytesAdded -> 3548, numTargetBytesRemoved ->
0, numTargetDeletionVectorsAdded -> 0, numTargetRowsMatchedUpdated -> 0,
```

```
executionTimeMs -> 4026, numTargetRowsInserted -> 2, numTargetRowsMatchedDeleted
-> 0, numTargetDeletionVectorsUpdated -> 0, scanTimeMs -> 2625,
numTargetRowsUpdated -> 0, numOutputRows -> 2, numTargetDeletionVectorsRemoved ->
0, numTargetRowsNotMatchedBySourceUpdated -> 0, numTargetChangeFilesAdded -> 0,
numSourceRows -> 2, numTargetFilesRemoved -> 0,
numTargetRowsNotMatchedBySourceDeleted -> 0, rewriteTimeMs -> 372}|NULL
|Apache-Spark/3.5.8 Delta-Lake/3.2.0|
|0      |2026-06-10 09:39:00|NULL  |NULL    |WRITE    |{mode -> Overwrite,
partitionBy -> []}
|NULL|NULL    |NULL    |NULL      |Serializable |false      |{numFiles -> 2,
numOutputRows -> 5, numOutputBytes -> 3552}
|NULL      |Apache-Spark/3.5.8 Delta-Lake/3.2.0|
+-----+-----+-----+-----+-----+-----+
-----
-----
-----+-----+-----+-----+-----+-----+
-----
-----
-----
-----
-----
-----
-----+-----+-----+
-----+
```

Versión 0 (original):

id_transaccion	id_cliente	producto	cantidad	precio	fecha
TXN003	C001	Teclado	1	120.0	2024-01-17
TXN004	C003	Monitor	2	450.0	2024-01-18
TXN005	C002	Webcam	1	80.0	2024-01-19
TXN001	C001	Laptop	1	2500.0	2024-01-15
TXN002	C002	Mouse	3	25.5	2024-01-16

Versión 1 (después del merge):

id_transaccion	id_cliente	producto	cantidad	precio	fecha
TXN003	C001	Teclado	1	120.0	2024-01-17
TXN004	C003	Monitor	2	450.0	2024-01-18
TXN005	C002	Webcam	1	80.0	2024-01-19

	TXN006	C004	Audífonos	2	60.0	2024-01-20
	TXN007	C005	Disco Duro	1	150.0	2024-01-21
	TXN001	C001	Laptop	1	2500.0	2024-01-15
	TXN002	C002	Mouse	3	25.5	2024-01-16
+-----+	+-----+	+-----+	+-----+	+-----+	+-----+	+-----+

7. Pipeline Completo: MinIO → Spark → PostgreSQL

Python

```
print("=" * 60)
print("PIPELINE END-TO-END: MINIO → SPARK → POSTGRESQL")
print("=" * 60)

# Leer datos transformados desde MinIO
df_final = spark.read.format("delta").load(delta_transformado)

# Configuración JDBC (misma del Notebook 03)
jdbc_url = "jdbc:postgresql://localhost:5432/electiva"
connection_properties = {
    "user": "postgres",
    "password": "123abc",
    "driver": "org.postgresql.Driver"
}

# Escribir a PostgreSQL
df_final.write.jdbc(
    url=jdbc_url,
    table="transacciones_procesadas",
    mode="overwrite",
    properties=connection_properties
)

print("Datos de MinIO escritos en PostgreSQL (tabla: transacciones_procesadas)")

# Verificar lectura desde PostgreSQL
df_verificacion = spark.read.jdbc(
    url=jdbc_url,
    table="transacciones_procesadas",
    properties=connection_properties
)

print("\nVerificación - Datos en PostgreSQL:")
df_verificacion.show()
```

```
# Conteos cruzados
count_minio = df_final.count()
count_pg = df_verificacion.count()

print(f"\nConteo en MinIO: {count_minio}")
print(f"Conteo en PostgreSQL: {count_pg}")
print(f"Integridad verificada: {'CORRECTA' if count_minio == count_pg else 'ERROR'}")
```

Salida esperada:

Shell

```
=====
PIPELINE END-TO-END: MINIO → SPARK → POSTGRESQL
=====
```

Datos de MinIO escritos en PostgreSQL (tabla: transacciones_procesadas)

Verificación - Datos en PostgreSQL:

id_transaccion	id_cliente	producto	cantidad	precio	fecha	total_linea	categoria	fecha_procesamiento
TXN001	C001	Laptop	1	2500.0	2024-01-15	2500.0	Alto Valor	2026-06-10
TXN002	C002	Mouse	3	25.5	2024-01-16	76.5	Bajo Valor	2026-06-10
TXN003	C001	Teclado	1	120.0	2024-01-17	120.0	Medio Valor	2026-06-10
TXN004	C003	Monitor	2	450.0	2024-01-18	900.0	Medio Valor	2026-06-10
TXN005	C002	Webcam	1	80.0	2024-01-19	80.0	Bajo Valor	2026-06-10

Conteo en MinIO: 5

Conteo en PostgreSQL: 5

Integridad verificada: CORRECTA

8. Detener Spark

Python

```
spark.stop()
print("\n NOTEBOOK 04 COMPLETADO")
```

Operaciones:

1. Verificación de MinIO con boto3
2. Escritura de Delta Lake en bucket S3 (MinIO)
3. Lectura y transformación desde MinIO
4. Time Travel y MERGE (UPSERT) en Delta Lake
5. Pipeline end-to-end: MinIO → Spark → PostgreSQL
6. Verificación de integridad de datos
7. Documentación de equivalencias cloud

Artefactos creados:

- Bucket: spark-lake
- Delta tables: transacciones, transacciones_transformadas
- PostgreSQL tables: transacciones_procesadas